TokenDance: Scaling Multi-Agent LLM Serving via Collective KV Cache Sharing

Zhuohang Bian
Peking University
Beijing, China
22373017@buaa.edu.cn

Feiyang Wu
Peking University
Beijing, China
2501111907@stu.pku.edu.cn

Chengrui Zhang
Peking University
Beijing, China
2301111796@stu.pku.edu.cn

Hangcheng Dong
Shanghai Jiao Tong University
Shanghai, China
donghangcheng@sjtu.edu.cn

Yun Liang
Peking University
Beijing, China
ericlyun@pku.edu.cn

Youwei Zhuo
Peking University
Beijing, China
youwei@pku.edu.cn

arXiv:2604.03143v1 [cs.DC] 3 Apr 2026

Abstract

Multi-agent LLM applications organize execution in synchronized rounds where a central scheduler gathers outputs from all agents and redistributes the combined context. This *All-Gather* communication pattern creates massive KV Cache redundancy, because every agent's prompt contains the same shared output blocks, yet existing reuse methods fail to exploit it efficiently.

We present TokenDance, a system that scales the number of concurrent agents by exploiting the All-Gather pattern for collective KV Cache sharing. TokenDance's KV Collector performs KV Cache reuse over the full round in one collective step, so the cost of reusing a shared block is paid once regardless of agent count. Its Diff-Aware Storage encodes sibling caches as block-sparse diffs against a single master copy, achieving 11–17× compression on representative workloads. Evaluation on GenerativeAgents and AgentSociety shows that TokenDance supports up to 2.7× more concurrent agents than vLLM with prefix caching under SLO requirement, reduces per-agent KV Cache storage by up to 17.5×, and achieves up to 1.9× prefill speedup over per-request position-independent caching.

ACM Reference Format:
Zhuohang Bian, Feiyang Wu, Chengrui Zhang, Hangcheng Dong, Yun Liang, and Youwei Zhuo. 2026. TokenDance: Scaling Multi-Agent LLM Serving via Collective KV Cache Sharing. In . ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Large language models (LLMs) are the execution engine of many multi-agent platforms for coding [8, 29], search [14, 15, 32], and simulation [16, 26–28]. Social simulation pushes multi-agent serving to its limits: it requires many agents running simultaneously, complex multi-round interactions where each agent's context grows over time, and tool use that further extends prompt length. As these applications grow from a handful of agents to tens or hundreds in production deployments, GPU memory becomes the dominant scaling constraint. Each agent maintains a KV Cache that persists across interaction rounds, and all active agents' caches must coexist in GPU memory simultaneously. The central challenge for multi-agent LLM serving is therefore maximizing the number of agents that can run concurrently under a fixed memory budget.

This memory pressure has a structural root cause. Multi-agent frameworks such as OpenClaw [24] and MoltBook [23] organize execution in synchronized rounds: each agent produces an output, a central scheduler gathers all outputs, and redistributes the combined context to every agent for the next round. We call this recurring data flow the **All-Gather** pattern, after the corresponding collective in distributed computing. The pattern directly explains the memory problem. As Figure 1 shows, each agent's prompt for round t+1 combines a private history (H) with the full set of shared output blocks (O). Because private histories differ in length, the same shared blocks land at different absolute positions across prompts, preventing the serving engine from recognizing the overlap. The result is N near-identical KV Caches in GPU memory. Therefore, memory grows with agent count rather than with the amount of unique context.

Figure 2 quantifies the scale of this redundancy. On an 80 GB A100 GPU serving Qwen2.5-14B, we compare 10 multi-agent sessions (250 agent subrequests total) against 250 independent single requests. The multi-agent workload consumes 41.5 GiB of KV Cache storage—99.3% of the pool—while independent requests use only 24.8 GiB (59.2%). The latency impact is immediate: multi-agent sessions hit a P99 of 136 s because the memory pool saturates from the start, whereas independent requests start lower and rise gradually to 125 s. The difference is that independent requests free their KV Caches after completion, whereas multi-agent KV Caches must coexist across rounds, saturating the pool and forcing the scheduler to preempt and swap.

Existing serving systems do not exploit this redundancy because they operate at the wrong granularity. Systems like vLLM [12] and SGLang [40] treat each agent subrequest as an independent inference request, applying prefix caching or position-independent caching per request. Agent-aware

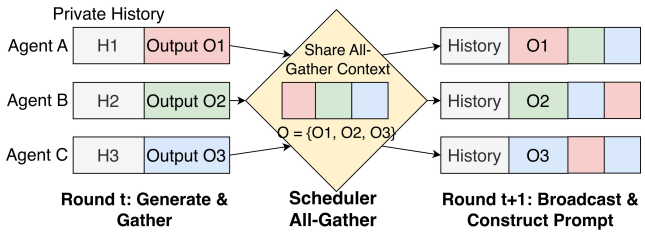
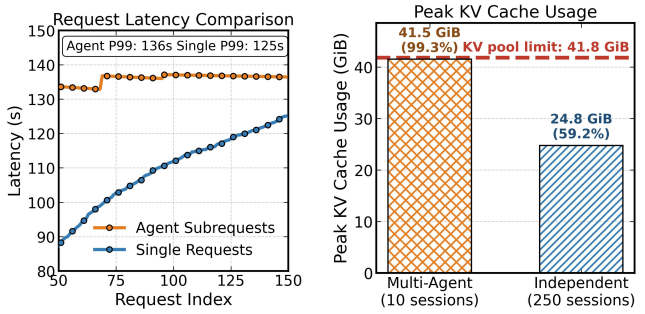


Figure 1. The All-Gather prompt structure. All agents receive the same output blocks (O), but the blocks appear at different positions because each prompt has its own private history (H) and may use a different block order. This structure arises in any multi-agent application that follows the All-Gather pattern.



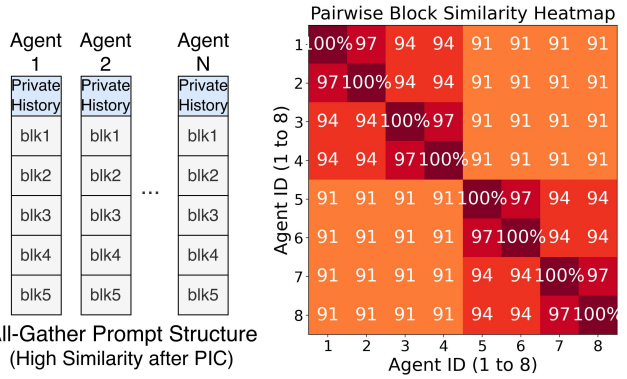
(a) Subrequest latency against request index.

(b) Peak KV Cache usage for the two workloads.

Figure 2. The scaling gap between multi-agent and independent workloads on a single A100-80GB GPU serving Qwen2.5-14B. Both workloads issue the same total number of subrequests (250), but the multi-agent workload nearly exhausts the KV Cache pool because each agent retains its own copy of the shared context across rounds, whereas independent requests free memory after completion.

schedulers such as Parrot [17], Autellix [22], and Tokencake [2] optimize *when* each subrequest runs and manage offloading, but do not change *how* the resulting KV Caches are computed or stored; every agent still holds a full, independent cache copy. The missing capability is round-level KV Cache optimization: detecting that agents in the same round share most of their context and using this overlap to reduce both compute and memory cost. Exploiting this overlap, however, requires solving two specific problems.

The first problem is *inefficient reuse*. Each agent's prompt in a new round contains a private history and a shared set of output blocks from the previous round. Because histories differ in length and the scheduler may choose different block orders, the same shared blocks appear at different absolute positions across requests. As Figure 1 shows, block O_1 starts at a different offset in each agent's prompt. Prefix caching [12, 40] only matches tokens from position zero, so it cannot detect any sharing once private histories diverge. Position-independent caching (PIC) methods [10, 34–36] remove this constraint and can reuse blocks at arbitrary offsets, but they still process each request separately: in an N-agent round the same shared blocks trigger N separate reuse passes, paying per-request overhead for what is fundamentally a round-level sharing opportunity. Section 2.3 and Figure 4 contrast this per-request approach with collective reuse.



(a) Schematic: Structural Overlap

(b) Measurement: Pairwise Block Similarity

Figure 3. High similarity of KV Caches after PIC reuse. Because all agents reuse the same shared blocks, their KV Caches differ only at the privately-recomputed positions.

The second problem is *inefficient storage*. In All-Gather workloads where agents share the majority of their round context, the resulting KV Caches after reuse are nearly identical. The reason is structural: the shared output blocks are reused from the same cached source, so they produce identical KV Cache values across all agents; the only entries that differ are tokens that each agent recomputes from its own history, which typically account for a small fraction of the total sequence. Figure 3 confirms this quantitatively: in an 8-agent GenerativeAgents [26] round, pairwise block similarity ranges from 91% to 97%, meaning the vast majority of cached data is duplicated across agents. Storing one full KV Cache per agent therefore wastes most of the GPU memory budget and directly caps how many agents can remain active. Making one round faster does not solve the problem if each agent still holds a full KV Cache copy. The per-agent storage cost must also shrink so the same hardware can sustain more concurrent agents.

We present TokenDance, a system that scales the number of concurrent agents on local GPUs by exploiting the All-Gather pattern for collective KV Cache sharing. Instead of optimizing individual requests, TokenDance operates on the multi-agent round as a whole. It amortizes reuse computation across all agents in a round so the cost of sharing a block is paid once regardless of agent count, and it compresses KV

Cache storage to only the inter-agent differences so memory cost grows with the actual difference rather than with the number of agents.

We evaluate TokenDance on the multi-agent social simulation frameworks GenerativeAgents [26] and AgentSociety [27]. Compared to vLLM with prefix caching, TokenDance reduces end-to-end latency by up to 2.3× and KV Cache storage by 94%. Against CacheBlend, TokenDance achieves a 1.9× speedup in the prefill phase. These gains translate directly into higher agent capacity: under the same latency target, TokenDance supports up to 2.7× more concurrent agents.

2 Background

2.1 The All-Gather Pattern

As introduced in Section 1, the All-Gather pattern organizes multi-agent execution in synchronized rounds where a scheduler gathers all agent outputs and redistributes the combined context. We now formalize this structure. For a system with N agents, each agent i maintains a private history H_i^t containing its system prompt and prior interactions up to round t. In round t, each agent j produces an output block O_j^t. The shared set of round outputs is:

$$O^t = \{O_1^t, O_2^t, \ldots, O_N^t\} \tag{1}$$

The prompt for agent i in round $t+1$ is then:

$$P_i^{t+1} = H_i^t \| \Pi_i(O^t) \tag{2}$$

where Π_i is the scheduler-defined layout of the shared output blocks for agent i. Every prompt in the round contains the same shared output blocks and one private history, and the shared blocks appear at different absolute positions across requests because the private histories differ in length.

2.2 KV Cache Reuse in LLM Serving

LLM serving is memory-bound: the system stores KV Caches to avoid recomputing attention keys and values for prior tokens, and managing these caches is the primary challenge for scaling system capacity. Existing reuse methods fall into two categories.

Prefix caching, used by vLLM [12] and SGLang [40], reuses cached tensors only when a new request shares an exact token prefix with a stored sequence. This works well for requests that share a common system prompt but fails once per-agent private histories cause the prefix to diverge.

Position-independent caching (PIC) methods [10, 34–36] remove this constraint by recovering shared blocks at arbitrary offsets. The key idea is to correct for positional encoding differences: a PIC method first applies RoPE rotation to align cached Keys with the target positions in the new request, then computes key differences between the rotated cached values and freshly computed values to identify *important positions*—token positions where the two differ significantly. Only these important positions are selectively recomputed, while the remaining positions reuse the rotated cached values directly. This recovers sharing beyond the prefix at the cost of additional compute for rotation and recomputation, and a minor accuracy trade-off at positions that are reused without recomputation.

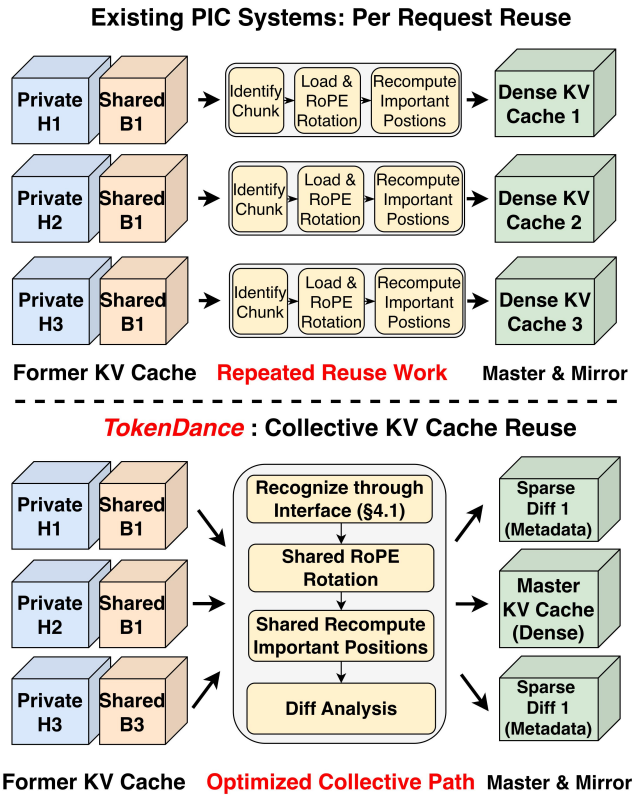


Figure 4. Per-request PIC reuse (top) vs. TokenDance's collective reuse (bottom). Existing PIC methods process each agent's shared blocks independently, repeating RoPE rotation and important-position selection N times. TokenDance groups the N requests and performs these operations once for the round.

2.3 Limitations of Existing Serving Systems

The All-Gather pattern creates a specific combination of redundancy that existing serving systems fail to exploit. We group prior work into two categories and explain why each falls short.

Agent-Aware, Compute-Centric Schedulers. Parrot [17], Autellix [22], and Teola [31] inject application-level dependency graphs into the serving backend to prioritize critical requests and reduce head-of-line blocking. ScaleSim [25] estimates future invocation distances to guide prefetching in simulation workloads. These systems decide *when* each request runs, but they do not control *how* the resulting KV Caches are computed or stored. The memory allocator underneath still treats every agent's cache as an independent

object. A burst of non-critical agents can therefore evict a critical agent's cache regardless of the schedule. None of these systems detects that the N requests in an All-Gather round share the same output blocks, so neither the reuse work nor the storage footprint is reduced.

KV-Cache-centric, Agent-Agnostic Engines. vLLM [12] and SGLang [40] reuse cached tensors through prefix matching, which breaks as soon as private histories diverge from position zero. CacheBlend [35] and EPIC [10] lift this positional constraint by reusing KV Cache blocks at arbitrary offsets, but they still analyze each request separately: N agents trigger N independent reuse passes over the same shared blocks. Mooncake [30] and CacheGen [19] expand the cache pool through disaggregation and compression, yet their policies are driven by global memory pressure, not by round structure. Tokencake [2] adds agent-aware scheduling on top of a KV-Cache-centric backend, proactively offloading stalled agents and reserving memory for critical ones. Across all of these systems, the final outcome is the same: every agent holds a dense, independent KV Cache copy, and no storage is freed even when over 90% of cache blocks are identical across agents in the same round.

Figure 4 contrasts this per-request approach (top) with the collective alternative that TokenDance introduces (bottom).

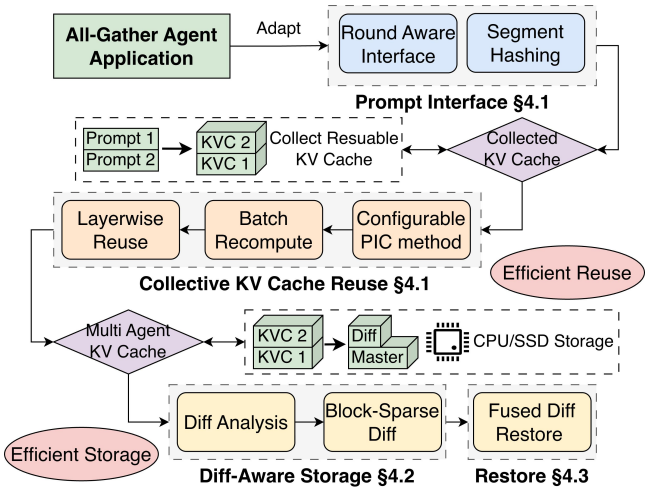


Figure 5. TokenDance Overview. A round-aware prompt interface preserves block boundaries so the runtime can identify shared content; collective KV Cache reuse amortizes the reuse cost across all agents in the round; diff-aware storage with fused restore compresses per-agent KV Caches to only the inter-agent differences.

3 Overview

TokenDance scales the number of active agents that a local GPU can support in an All-Gather system by addressing both redundant reuse computation and redundant KV Cache storage. The key design principle is to lift the unit of optimization from a single request to the All-Gather round: every operation that is common across agents in a round is performed once and shared, while only the per-agent differences are handled individually. Figure 5 illustrates the three components that realize this principle.

The round-aware prompt interface (Section 4.1) preserves the logical block structure of each prompt by inserting separator tokens between adjacent blocks, so the runtime can recognize shared blocks even when they appear at different absolute positions. The collective KV Cache reuse algorithm (Section 4.2) exploits this structure to perform reuse for a compatible group of requests in one pass, paying the reuse overhead once per round instead of once per request. The diff-aware storage scheme (Section 4.3) encodes sibling caches as sparse differences against a shared Master copy, and the fused restore path (Section 4.4) applies those differences during GPU transfer to avoid a separate reconstruction step. Together, these components reduce both the compute work and the storage cost that each additional agent adds, allowing the same hardware to sustain more agents at the same service level.

4 TokenDance Design

The design of TokenDance starts from a mismatch between the All-Gather application structure and the serving system. Multi-agent applications communicate in rounds, but the serving system receives a flat token stream, loses the logical boundary between private and shared blocks, and treats every request as independent.

This mismatch creates three compounding inefficiencies. First, the runtime cannot identify shared blocks across requests because position-based indexing conflates content identity with absolute offset. Second, position-independent reuse methods repeat the same analysis for each request in the round, even though all requests consume the same shared blocks. Third, even after reuse, the resulting KV Caches are nearly identical, yet the system stores one full dense copy per request. The cumulative result is an inflated per-agent KV Cache cost that directly caps the number of concurrent agents.

TokenDance addresses these inefficiencies with two design rules. First, it keeps the round structure visible until the runtime can use it. Second, it preserves one correct cache state per request while sharing the work and state that are truly common across the round. The first rule leads to a round-aware interface (Section 4.1) and collective KV Cache reuse (Section 4.2). The second rule leads to diff-aware storage and fused diff restore (Section 4.4). The rest of this section describes each component in detail.

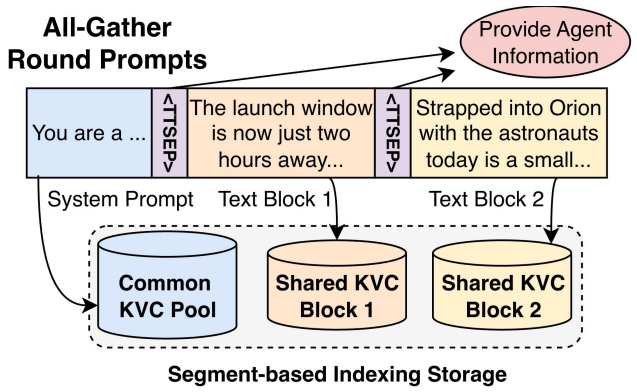


Figure 6. Example of TokenDance's round-aware prompt interface. Each prompt is composed of a private history block and a shared set of output blocks, with reserved separator tokens (<TTSEP>) in between.

4.1 Round-Aware Prompt Interface

The runtime cannot exploit the All-Gather pattern if it only receives a flat token stream. TokenDance provides a round-aware prompt interface that preserves the logical block structure of each prompt, so the runtime can recognize shared blocks even when they appear at different absolute positions across requests.

Any multi-agent framework that follows the All-Gather pattern can adopt this interface with minimal code changes. If the application does not follow the pattern, TokenDance falls back to the standard single-request path with no performance loss.

The application assembles one prompt per agent and inserts a reserved separator token <TTSEP> between adjacent logical blocks. Figure 6 shows an example for three agents in one round. Each prompt contains a private history block and the same set of shared output blocks, with separator tokens in between. The delimited blocks make the reusable relationship between requests explicit: the private history, each shared agent update, and the round task are all individually identifiable after tokenization.

Once block boundaries are visible, the runtime switches from fixed-size chunk hashing to segment-based hashing. Each shared update is indexed by its own content segment rather than by its absolute position in one long prompt. Two requests that contain the same shared update therefore map that update to the same cache object, even when their private histories differ in length. This is the first step that turns the All-Gather pattern into a serving optimization: the runtime now knows which blocks are shared and can reason about reuse at the block level rather than the prefix level.

4.2 Collective KV Cache Reuse

Once the round structure is visible, the next problem is repeated reuse work. Existing position-independent caching (PIC) methods [10, 34–36] can recover sharing beyond the prefix, but they still process each request independently. In an N-agent round, the runtime performs N separate reuse passes for the same set of shared blocks. Each pass applies RoPE, computes key differences, and selects important positions—all operations that produce nearly identical results across requests because the shared blocks are content-identical. This per-request redundancy is a poor match for the All-Gather pattern, where the shared round updates are common to every request in the round.

TokenDance resolves this redundancy with collective KV Cache reuse, illustrated in Figure 7. Rather than processing each request independently, the KV Collector groups the N requests in a round and performs one shared RoPE rotation and one shared important-position selection pass, then updates only the differing positions in each request's cache. Figure 7 contrasts this collective path (T3) with full recomputation (T1) and per-request PIC reuse (T2).

Grouping Compatible Requests. The KV Collector collects requests from the same All-Gather round whose prompt spans are compatible for collective processing. Compatibility requires that the requests have the same active prompt length, the same cached span visible to the cache layer, and non-overlapping slot mappings in the execution engine. These are execution constraints that ensure lockstep layerwise processing; requests that do not satisfy them fall back to separate groups or to the single-request path.

Layerwise Collective Reuse. For each compatible group, the runtime drives layerwise retrieval and model execution in lockstep. At each layer, KV Collector concatenates the Q and K tensors from all requests in the group into one combined tensor and applies a single batched RoPE call. On the configured check layer, the runtime compares the rotated Keys against the cached Keys for the entire group in one batched difference pass. This pass identifies the important positions—the token positions where the cached values diverge significantly from the freshly computed values—for each request simultaneously. The runtime then refreshes only those important positions in each request's cached K and V tensors. Later layers reuse the important-position set identified on the check layer directly, without repeating the difference computation.

The expensive operations—RoPE rotation and key-difference analysis—are therefore performed once for the group rather than once per request. Only the final cache update remains request-specific, because each request's private history produces different values at the positions that need refreshing. In a request-centric path, the reuse-analysis overhead grows linearly with the number of agents. In TokenDance, the group pays one shared RoPE pass and one shared diff-analysis pass per layer, and only the per-position refresh scales with agent count. As the number of agents in a round grows, this amortization reduces the total reuse

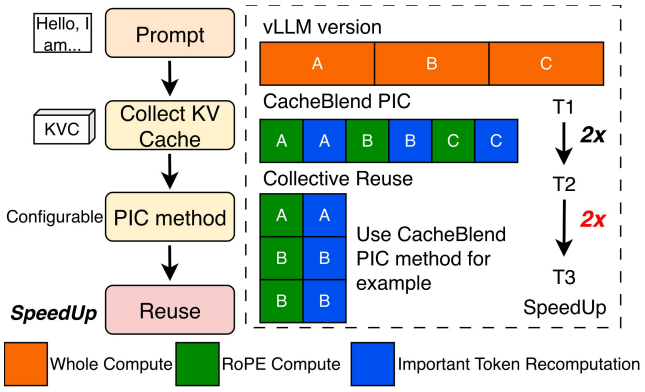


Figure 7. Collective KV Cache reuse for a three-agent All-Gather round. Left: each agent's prompt contains a private section and the same shared blocks in different orders. Right: vLLM computes all three from scratch (T1); per-request PIC processes each request independently (T2); TokenDance groups them and shares the RoPE and important-position selection work across the group (T3), paying the reuse overhead once per round.

work per round. That reduction is the compute side of the scaling gain.

The collective amortization is decoupled from the underlying per-position recovery method. The current prototype uses CacheBlend [35]'s selective recomputation as the default backend, but any PIC method that accepts a set of token positions and returns corrected K/V tensors can serve as a drop-in replacement through an adapter interface.

Reuse Plan Output. Collective KV Cache reuse also produces metadata that Diff-Aware Storage later consumes. The reuse path records the group membership, the accumulated per-request deviation scores, and the request chosen as the Master. The Master is not semantically special. It is simply the request whose recovered result is closest to the group's common structure, typically the one with the lowest total deviation from the shared blocks. Selecting a good Master minimizes the size of the sparse corrections that Diff-Aware Storage must keep for the remaining requests. The reuse plan therefore serves as the bridge between the compute optimization in this subsection and the storage optimization in the next.

4.3 Diff-Aware Storage

Collective KV Cache reuse removes redundant compute, but it does not remove redundant storage. After reuse completes for one round, the system still holds one dense KV Cache per request. This is wasteful because the requests in one All-Gather round share most of their logical blocks and differ only in a private history and a small number of position-sensitive updates. If the system stores every recovered result in full, it merely shifts redundancy from prefill compute to memory capacity.

This problem is especially acute for multi-agent serving because rounds repeat and agent states stay live across rounds. In a system with N agents running over R rounds, storing one full cache per agent per round leads to memory consumption that grows as $O(N \times R)$. If the system stores many near-duplicate KV Caches, memory pressure rises quickly and the number of active agents falls. Storage must therefore exploit the same All-Gather structure that reuse exploits: the requests in a round are structurally similar and differ only at a predictable subset of positions.

Master-Mirror Layout. TokenDance uses a Master-Mirror layout to compress the per-round cache family, as shown in Figure 8. Diff-Aware Storage keeps one dense cache as the Master and stores each remaining cache as a Mirror that contains only the difference from that Master. The adapter forwards the reuse plan from collective KV Cache reuse into the store path, so storage already knows which request should become the Master and which positions differ in each Mirror. When no explicit reuse plan is available—for example, when a request arrives outside a recognized All-Gather round—the backend falls back to a token-similarity heuristic to find a reusable Master among existing cached entries.

Block-Sparse Diff Representation. The Mirror correction is stored as a block-sparse K/V diff. The differences among recovered caches are concentrated in a subset of token positions whose values changed because of private context or position-dependent RoPE rotation. These differing positions tend to cluster in contiguous blocks that correspond to the private history segment or to the boundary regions between shared blocks. A block-granular representation captures this clustering with much lower metadata cost than a fine-grained element-wise diff. It also aligns naturally with the block-based memory management used by modern serving engines and with the tile-aligned restore pipeline described in Section 4.4.

On read, the storage layer does not eagerly rebuild a dense Mirror tensor. Instead, it returns a lightweight mirror object that keeps a reference to the Master and the sparse diff metadata. This preserves a standard logical KV Cache abstraction for the caller while delaying materialization until the runtime actually needs the data. The Mirror occupies only 10–20% of a full cache, so the system saves memory immediately without paying the restore cost until execution time.

The strength of this layout depends on how much of the prompt is shared versus private. The All-Gather pattern is a natural fit because the shared output blocks from the previous round dominate the prompt, and only the private history and round task are unique per agent. If requests diverge more strongly—for example, in a system where each agent receives a substantially different subset of the shared outputs—then

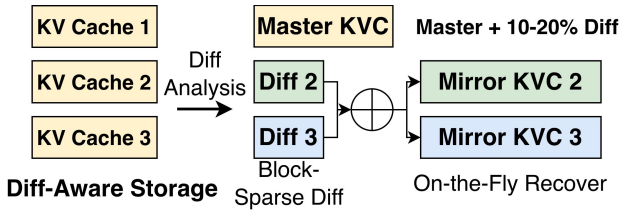


Figure 8. Diff-aware storage with the Master-Mirror layout. Left: after reuse, the KV Caches of three agents differ at only 10–20% of positions. Right: Diff-Aware Storage stores one full Master cache and encodes each remaining cache as a sparse diff (Diff 2, Diff 3). On restore, the system reconstructs each Mirror on the fly from the Master plus its diff.

Algorithm 1 Fused Diff Restore for One Mirror Request

Require: Mirror M, slot map S, positions P^{old}, P^{new}, layers L.

1: Allocate ping-pong buffers B^{load}, B^{comp}.
2: **for** $\ell \leftarrow 1$ **to** L **do**
3: Load Master chunks of layer ℓ into B^{load}.
4: Sync; swap $B^{load} \leftrightarrow B^{comp}$.
5: $D \leftarrow \textsc{GetDiff}(M, \ell)$
6: **if** $D \neq \varnothing$ **then**
7: Update $B^{comp}.\{K, V\}$ at $D.indices$ with $D.values$.
8: **end if**
9: $B^{comp}.K \leftarrow \textsc{RoPERecover}(P^{old}, P^{new}, B^{comp}.K)$
10: Write B^{comp} to paged KV Cache at layer ℓ using S.
11: **end for**

the Mirror corrections grow larger and the storage benefit diminishes. We quantify this relationship in Section 6.4.

4.4 Fused Diff Restore

Compression is useful only if restore stays cheap on the critical path. A naive Mirror restore would load the full Master, copy it into a new buffer, and overwrite the differing blocks—adding an extra dense write-then-read round trip for an object the system never keeps.

TokenDance eliminates this overhead by applying the sparse corrections inside the layerwise transfer path that already moves cached KV data into paged GPU memory. Algorithm 1 shows the procedure. Two GPU buffers alternate roles in a ping-pong fashion: one receives Master chunks from storage while the other undergoes in-place correction and writeback. At each layer, the path applies the block-sparse diff (line 7), recovers RoPE positions (line 9), and writes the result into paged KV Cache memory (line 10)—all within the same pass that ordinary cache loads already perform. No separate dense Mirror is ever materialized.

The only additional work per layer is the correction at line 7, whose cost is proportional to the number of differing blocks—typically 10–20% of the total. Figure 9 illustrates the

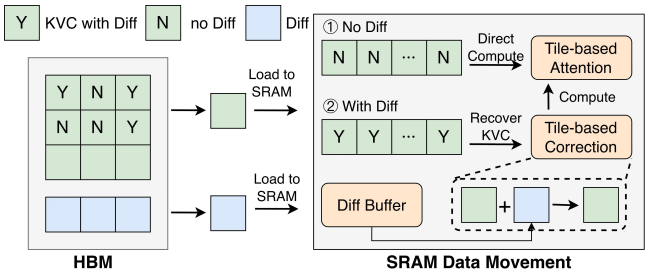


Figure 9. Fused diff restore at block granularity. Blocks marked "Have Diff" are corrected in SM memory before attention; blocks with no diff bypass the correction path.

block-level dispatch. Blocks identical to the Master proceed directly to attention. Blocks that carry diffs are corrected in SM memory before being fed to FlashAttention. The block-sparse format makes this skip-or-correct decision per block without scanning the full cache, and the block size aligns with attention tile sizes so corrected blocks require no additional reshaping.

The current prototype fuses the diff into the transfer pipeline before attention, not inside the attention tile loader. This already removes the dense reconstruction step and reduces extra memory traffic on the critical path. A deeper fusion—applying diffs as attention tiles are loaded from HBM into shared memory—is a natural extension.

Taken together, diff-aware storage and fused restore address the memory side of the scaling problem. Storage reduces how much state each additional agent contributes to the memory footprint. Fused restore ensures this compression remains usable during online execution rather than only at rest. This is how TokenDance turns the structural similarity inherent in the All-Gather pattern into higher active-agent capacity on local GPUs.

5 Implementation

TokenDance is implemented on top of LMCache and vLLM. We add approximately 3K lines of Python and 500 lines of CUDA/C++ across four parts: round-aware segment indexing, collective KV Cache reuse, diff-aware storage, and fused sparse restore.

Round-Aware Segment Indexing. The application inserts <TTSEP> separators between logical blocks as described in Section 4.1. On the runtime side, we replace the fixed-size chunk hash table with a segment-based hash table that splits prompts at separator boundaries and indexes each segment independently.

Collective KV Cache Reuse. The vLLM V1 adapter scans requests in each scheduler step, and dispatches each group to the collective path described in Section 4.2. Internally, the KV Collector creates one compute generator and one retrieval generator per request and drives them in lockstep across layers.

Diff-Aware Storage. We add a diff-aware backend that wraps the normal LMCache storage backend. When a reuse plan is available, the store path uses it to select the Master and identify Mirror positions directly. When no reuse plan is available, the backend falls back to a token-similarity heuristic. Master chunks are written unchanged. Each Mirror chunk is serialized as a block-sparse K/V diff: the backend records the touched block indices and the K/V correction values for those blocks. When both K and V planes touch the same blocks, the implementation shares the block-index list between them to reduce metadata size. On read, the backend returns a lightweight mirror object rather than a dense tensor, deferring materialization to the restore path.

Fused Sparse Restore. We extend the GPU connector's layerwise ping-pong transfer path to recognize mirror objects. The connector loads Master chunks into its temporary buffer, merges the sparse diff metadata for the current layer, and invokes a paired K/V diff kernel in place before RoPE recovery and paged-memory writeback. The CUDA extension provides two kernels: one for a single KV plane and one that updates K and V together. The paired kernel is used when the sparse metadata is well aligned; otherwise the connector falls back to dense restore. As discussed in Section 4.4, fusion currently occurs in the transfer pipeline before attention, not inside the attention tile loader.

6 Evaluation

The main question of our evaluation is not whether TokenDance speeds up one round, but whether it can scale the number of active agents under the same service target. We organize the evaluation around five questions: (Q1) the system-level scaling benefit (§6.2); (Q2) where this benefit comes from on the compute side (§6.3); (Q3) where it comes from on the memory side (§6.4); (Q4) whether the fused retrieval path preserves these gains on the critical path (§6.5); and (Q5) whether TokenDance preserves model output fidelity (§6.6).

6.1 Setup and Metrics

We evaluate TokenDance on NVIDIA A100 80 GB GPU. We use two models, Qwen2.5-7B and Qwen2.5-14B, to show how gains change with model size. We replay traces from GenerativeAgents and AgentSociety, two representative All-Gather-pattern frameworks that span different operating regimes: GenerativeAgents uses shorter private histories and fewer agents per round, while AgentSociety uses longer histories with more agents.

We compare against three baselines that span the space of existing reuse strategies: (1) vLLM with prefix caching, which represents the standard request-local reuse path; (2) CacheBlend without per-request KV Cache recovery (`CacheBlend Ordinary Path`), which uses a CPU-side KV Cache pool but no cross-prefix reuse; and (3) CacheBlend with full per-request KV Cache recovery, the most widely adopted open-source PIC method [35], which performs selective cross-prefix reuse without round-level sharing.

Our metrics reflect the scaling goal directly. We derive two capacity metrics from round latency (the end-to-end time to serve one agent round), peak KV Cache footprint, and restore overhead. The first is the maximum number of agents sustained below a fixed latency target (1500 ms). The second is the maximum number of agents sustained at a given QPS before latency exceeds that target. These two views correspond to the two ways a serving system is provisioned: by latency SLO and by throughput budget.

6.2 Main Result: Scaling the Number of Active Agents

Under the same latency target or QPS target, how many agents can each system support? For each system, we sweep the number of agents from 1 to 10 and the offered QPS from 1 to 16, recording round latency at each operating point. From these sweeps, we derive the two capacity views described above.

Figure 10 presents the results across four configurations (two workloads × two models). Each configuration shows two panels: round latency against agent count at $QPS = 10$ (left, with a dashed 1500 ms SLO line) and maximum supported agents against QPS (right).

TokenDance consistently achieves the lowest round latency across all configurations and agent counts. On GenerativeAgents/Qwen2.5-7B, TokenDance keeps round latency below the SLO for all 10 agents, while vLLM with prefix caching and CacheBlend Ordinary Path cross the SLO at 10 agents. On GenerativeAgents/Qwen2.5-14B, TokenDance still sustains all 10 agents below the SLO, while vLLM exceeds it at 7.5 agents and both CacheBlend variants exceed it at similar points. The gap widens with model size because the per-agent KV Cache footprint grows, and TokenDance's deduplication provides proportionally larger savings.

The supported-agents panels quantify the scaling benefit. On GenerativeAgents/Qwen2.5-7B at $QPS = 16$, TokenDance supports 8 agents while CacheBlend drops to 4 and vLLM drops to 2. On GenerativeAgents/Qwen2.5-14B, TokenDance maintains 4 agents across the full QPS range, while vLLM drops to 1 agent beyond $QPS = 8$. The AgentSociety workload (bottom row) shows the same pattern: at $QPS = 4$ on AgentSociety/Qwen2.5-7B, TokenDance supports 4 agents while vLLM and CacheBlend Ordinary Path support 3. On AgentSociety/Qwen2.5-14B, the baselines can barely sustain a single agent beyond $QPS = 4$, whereas TokenDance still supports 2 agents at $QPS = 16$.

Two trends are worth highlighting. First, TokenDance's advantage grows with agent count. At small agent counts (1–2), all systems perform similarly because there is little cross-agent redundancy to exploit. As the agent count rises, the baselines pay linearly growing reuse and storage costs,

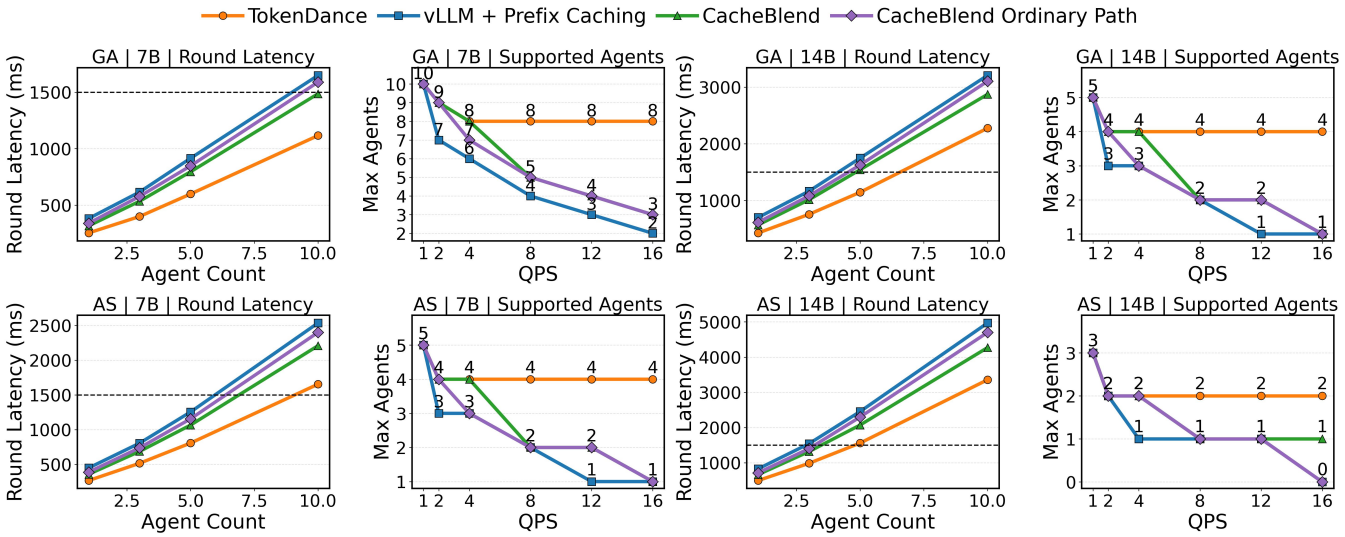


Figure 10. Scaling capacity overview across two workloads (GenerativeAgents, AgentSociety) and two models (Qwen2.5-7B, Qwen2.5-14B). Left panels: round latency vs. agent count at $QPS = 10$; the dashed line marks the 1500 ms SLO. Right panels: maximum number of agents that stay below the SLO at each QPS level. TokenDance (orange) consistently supports more agents than all baselines across the full QPS range.

while TokenDance amortizes both across the round. Second, TokenDance's advantage is more pronounced on the 14B model than on the 7B model. This is because the per-agent KV Cache footprint doubles with model size, making deduplication more valuable in absolute terms. These results confirm the central claim: TokenDance increases the number of concurrently active agents by exploiting the All-Gather pattern, and the benefit scales with both agent count and model size.

6.3 Why Scale Improves: Collective Reuse Compute

Does collective KV Cache reuse amortize the reuse overhead that grows with agent count in request-centric methods? The scaling result in Q1 combines compute and memory gains. We now isolate the compute contribution by measuring how collective KV Cache reuse affects prefill throughput as agent count grows.

We replay a single GenerativeAgents round while varying the number of agents (3, 5, 10, 15, and 20) and the offered QPS (1, 2, 4, 8, 12, and 16). All agents share the same set of output blocks from the previous round, so the degree of cross-agent redundancy is controlled. We measure the speedup of TokenDance's collective KV Cache reuse over the serial baseline, where each request is processed by the PIC backend independently.

Figure 11 reports the results. The y-axis shows the speedup of collective reuse over the serial path; a value above 1.0 means collective reuse is faster. The peak speedup is 2.57×, achieved at 10 agents and $QPS = 1$. At low QPS, larger groups benefit more from amortization: the 10-agent configuration reaches 2.57× and the 5-agent configuration reaches 1.31× at $QPS = 1$, while the 3-agent configuration achieves a more modest 1.17×. The 15-agent and 20-agent configurations reach 1.61× and 1.73× at $QPS = 1$ respectively; beyond 10 agents, scheduling overhead grows and partially offsets the per-request amortization gain.

Two observations stand out. First, all agent counts achieve speedup above 1.0 across the entire QPS range, confirming that collective reuse is consistently beneficial regardless of group size. The benefit is largest at low QPS where the system has time to fully exploit the collective grouping, and smallest at high QPS where GPU compute saturation limits the margin. Second, as QPS increases beyond 4, the speedup converges across agent counts to the 1.2–1.6× range. At high QPS, the GPU becomes compute-saturated and the bottleneck shifts from redundant reuse work to raw compute capacity. Nonetheless, even at $QPS = 16$, collective reuse still provides a 1.3–1.5× speedup, which directly translates to the higher agent capacity observed in Q1.

The key takeaway is that collective KV Cache reuse amortizes the RoPE and important-position selection cost across the round. The per-request overhead of reuse analysis grows sublinearly with agent count under TokenDance, whereas it grows linearly under request-centric methods like CacheBlend. This sublinear scaling is what shifts the latency curve downward in Figure 10 and allows more agents to fit under the SLO.

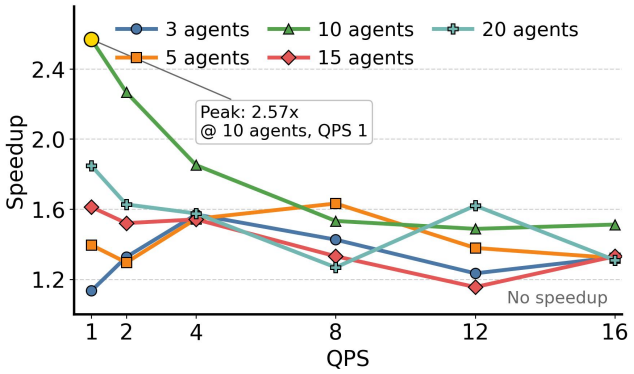


Figure 11. Collective KV Cache reuse speedup over serial (per-request) PIC recovery for varying agent counts (3, 5, 10, 15, 20) and QPS levels on the GenerativeAgents workload. The peak speedup of 2.57× occurs at 10 agents and $QPS = 1$. All configurations exceed 1.0× across the full QPS range, confirming that collective reuse is consistently beneficial.

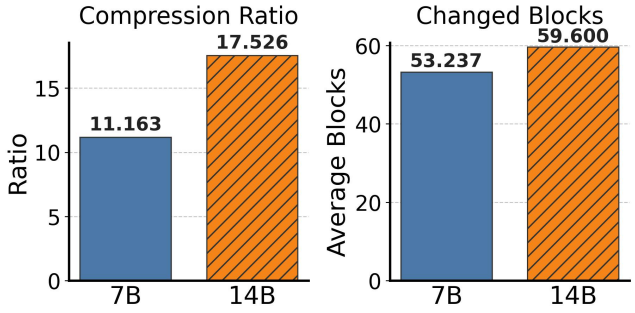


Figure 12. Redundancy characterization of recovered KV Caches across agents in a single GenerativeAgents round. Left: compression ratio (full cache size divided by Master-plus-diff size). Right: average number of 32-token blocks that differ between a Mirror and its Master. The 14B model achieves higher compression because cache tensors per token are larger while the number of differing blocks stays similar.

6.4 Why Scale Improves: KV Cache Footprint and Storage Redundancy

Does Master-Mirror storage significantly lower the KV Cache cost per agent round? Collective KV Cache reuse reduces compute waste, but the system still holds one recovered KV Cache per agent. If these caches remain dense and duplicated, memory pressure will still cap the number of active agents. We now isolate the memory contribution by characterizing the redundancy among recovered caches and measuring the storage savings of the Master-Mirror layout.

Redundancy characterization. We produce recovered KV Caches for all agents in a single GenerativeAgents round and compare each Mirror against the selected Master at block granularity (32 tokens per block). Figure 12 reports two metrics for both the 7B and 14B models. The left panel shows the compression ratio, defined as the ratio of the full dense KV Cache size to the Master-plus-diff size. The 7B model achieves a compression ratio of 11.2×, meaning that the diff-encoded Mirror is roughly 9% the size of a full cache. The 14B model achieves 17.5×, because the larger model produces longer cache tensors per token while the number of differing blocks remains similar. The right panel shows the average number of changed blocks per Mirror. The 7B model averages 53.2 changed blocks per Mirror, and the 14B model averages 59.6. These numbers are small relative to the total block count of a full cache (typically 500–700 blocks in these workloads), confirming that the vast majority of blocks are identical to the Master.

The higher compression ratio on the 14B model is significant for scaling. Each 14B agent's full KV Cache is approximately twice the size of a 7B agent's cache. By compressing Mirrors to about 6% of the full size, TokenDance effectively removes the model-size penalty from the per-agent storage cost. This explains why TokenDance's scaling advantage over the baselines is larger on the 14B model in Figure 10: the storage savings translate directly into more agents fitting within the GPU memory budget.

Implied capacity gain. The compression ratios above translate to concrete capacity improvements. Under the Master-Mirror layout, the KV Cache cost of N agents in one round is approximately $1 + (N-1)/R$ full caches, where R is the compression ratio. For $N = 10$ agents on the 7B model ($R = 11.2$), this is $1 + 9/11.2 \approx 1.8$ full caches instead of 10, a 5.6× reduction in KV Cache memory. For the 14B model ($R = 17.5$), the cost is $1 + 9/17.5 \approx 1.5$ full caches instead of 10, a 6.7× reduction. This memory reduction is the mechanism that allows TokenDance to sustain more agents within the same GPU memory budget.

6.5 End-to-End Retrieval Cost

Does the fused retrieval path preserve the storage gains during online serving, or does reconstruction latency erase the compression benefit? A naive restore path would reconstruct a dense Mirror by copying the full Master and overwriting the differing blocks, adding an extra write-then-read round trip on the critical path. We compare this dense restore approach against TokenDance's fused diff path, which applies sparse corrections during the layerwise GPU transfer pipeline without materializing a separate dense copy.

Figure 13 reports the results for 1, 3, 5, and 10 agents at QPS levels from 1 to 8 on the GenerativeAgents/Qwen2.5-7B configuration. The left panel plots absolute restore latency for both paths. Dashed lines show dense restore; solid lines show fused retrieval. At 10 agents and $QPS = 1$, dense restore takes 0.59 ms per Mirror, while fused retrieval takes 0.43 ms, a 27% reduction. At 1 agent, fused retrieval costs 0.13–0.18 ms,

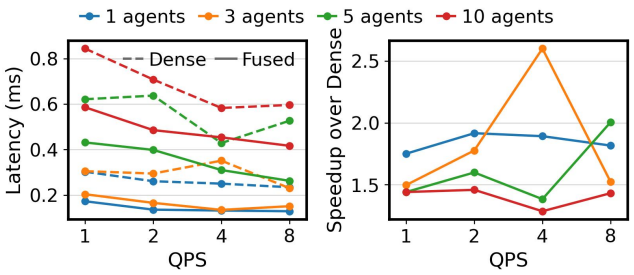


Figure 13. Latency analysis of Mirror state reconstruction on GenerativeAgents using Qwen2.5-7B. Left: absolute restore latency for dense reconstruction (dashed) and fused diff retrieval (solid) across agent counts and QPS levels. Right: speedup of fused retrieval over dense restore. Fused retrieval consistently reduces restore latency by 1.3–2.6× by avoiding a separate dense materialization step.

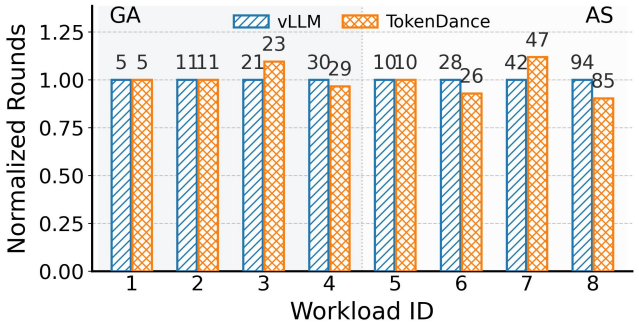


Figure 14. Accuracy evaluation across eight scenarios from GenerativeAgents (IDs 1–4) and AgentSociety (IDs 5–8). Each bar shows the number of simulation rounds completed before the first output divergence between TokenDance and vLLM (prefix caching, temperature = 0). The relative difference Δ is annotated above each pair. Workload ID mapping: 1 = Meet and Greet, 2 = Valentine's Day Party, 3 = Election Discussions, 4 = Winning the Election, 5 = Information Outbreak, 6 = Pre-Landfall Activity, 7 = Hurricane, 8 = Economic Stabilization. Three scenarios show zero divergence; the remaining differences are attributable to the underlying PIC method, not to TokenDance.

which is negligible compared to the overall round latency of several hundred milliseconds. As agent count increases, the absolute gap between dense and fused paths widens because fused retrieval avoids materializing increasingly large dense tensors.

The right panel reports the speedup of fused retrieval over dense restore. The speedup ranges from 1.3× to 2.6× depending on agent count and QPS. The 3-agent configuration achieves the highest peak speedup of 2.6× at $QPS = 4$. The 1-agent configuration shows a consistent 1.8–2.0× speedup across the QPS range. At 10 agents the speedup is 1.3–1.5× because the restore pipeline becomes partially bandwidth-bound regardless of the reconstruction strategy. Across all configurations, fused retrieval is consistently faster than dense restore, confirming that the storage compression from Q3 is preserved on the online critical path.

The combination of Q3 and Q4 closes the memory-side argument. Master-Mirror storage reduces the per-agent KV Cache footprint by 11–17× (Q3), and fused diff retrieval ensures that accessing the compressed representation adds less latency than reconstructing a dense copy (Q4). Together, these two mechanisms enable the higher agent counts observed in Q1.

6.6 Accuracy Influence

Does TokenDance's collective reuse and compressed storage alter model output compared to a standard per-request serving system under greedy decoding? By construction, TokenDance's collective path and Master-Mirror storage produce the same recovered KV Caches as the underlying PIC method applied per request; the collective grouping changes execution order but not the numerical result. In our evaluation we use CacheBlend as the PIC backend, so TokenDance produces the same output as CacheBlend with per-request recovery. Any output difference relative to a non-PIC baseline (e.g., vLLM with prefix caching) is therefore attributable to CacheBlend's selective recomputation, not to TokenDance.

To verify this, we run both TokenDance and vLLM with prefix caching on the same set of agent rounds, setting temperature to 0 to eliminate sampling randomness and using identical system prompts and input histories for every agent. For each scenario we record the number of simulation rounds that complete before the first divergence, i.e., the first round in which at least one agent produces a different response under the two systems. Figure 14 reports the results across eight scenarios spanning both workloads (IDs 1–4 from GenerativeAgents, IDs 5–8 from AgentSociety).

In three of the eight scenarios (Meet and Greet, Valentine's Day Party, and Information Outbreak), both systems produce identical output for the entire trace ($\Delta = 0.0\%$). In the remaining five scenarios, the round counts differ by 3.3%–11.9%. These differences arise because CacheBlend's PIC recovery introduces small numerical perturbations at selectively recomputed positions; under greedy decoding, such perturbations can eventually flip a single token choice, which then cascades through subsequent rounds. This is consistent with the stochastic nature of multi-agent simulation, where minor perturbations in one round propagate unpredictably through later rounds. The key result is that TokenDance does not introduce any additional accuracy degradation beyond what the underlying PIC method already produces.

7 Related Work

LLM Serving Systems. vLLM [12], SGLang [40], Orca [38], and Sarathi-Serve [1] improve batching, scheduling, and memory management for LLM inference. These systems optimize execution for individual requests but do not exploit the round-level KV Cache redundancy that TokenDance targets.

Request-Centric KV Cache Reuse. Prefix caching in SGLang [40] and vLLM [12] reuses cached state when a new request shares an exact prefix with a stored sequence. PromptCache [7] reuses cached modules identified by markup, and DroidSpeak [18] shares KV Caches across different LLMs. EPIC [10], KVLink [34], and KVComm [36] recover reuse at arbitrary positions through position correction, and CacheBlend [35] further adds selective recomputation to restore accuracy at important positions. All of these methods operate on one request at a time. TokenDance differs by treating the agent round as the reuse unit: it shares reuse work across all agents in a round and compresses the resulting set of KV Caches that this collective execution produces.

KV Cache Compression and Movement. HCache [5], InfiniGen [13], CacheGen [19], MoonCake [30], and CachedAttention [4] reduce the cost of long-context serving through recomputation, offloading, compression, or disaggregation. These methods target individual caches. TokenDance instead exploits similarity across sibling caches from the same round. The Master-Mirror layout and fused diff path are designed around cross-cache redundancy rather than single-cache compression alone.

KV Cache Eviction and Quantization. H2O [39] and ScissorHands [20] reduce KV Cache memory by evicting tokens with low attention scores, while StreamingLLM [33] and FastGen [6] keep only a fixed window plus a few important tokens for long-sequence serving. KIVI [21] and KVQuant [9] take a different path, quantizing cached values to 2–3 bits with little accuracy loss. These methods shrink individual caches but do not address cross-cache redundancy. TokenDance is orthogonal: it removes the duplicated data across sibling caches first, and per-cache compression can be applied on top.

Attention Kernels and Sparse Backends. FlashAttention [3] shows that attention performance depends heavily on data movement. FlashInfer [37] provides efficient variable-length and block-sparse attention backends. Our fused diff kernel follows the same data-movement logic. It aligns sparse diff blocks with attention tiles so compressed caches can be used without an extra dense reconstruction pass.

Multi-Agent LLM Systems. Parrot [17], Autellix [22], Teola [31], ScaleSim [25], and Tokencake [2] optimize scheduling, prefetching, or execution order for agent applications. LRAgent [11] shares KV Caches across multi-LoRA agents by decomposing caches into shared and adapter-specific parts. GenerativeAgents [26], AgentSociety [27], OpenClaw [24], and MoltBook [23] show that many real agent workloads run in explicit rounds with a shared scheduler. These systems define or schedule agent applications; TokenDance optimizes the KV Cache layer for the repeated gather-and-redistribute pattern that they all share.

8 Conclusion

We presented TokenDance, a system for scaling multi-agent LLM serving through collective KV Cache sharing. The design follows two core observations. Current reuse methods are inefficient for agent rounds, and even after round-level KV Cache reuse the resulting KV Caches remain highly compressible. TokenDance turns these observations into a progressive design with collective KV Cache reuse, Master-Mirror storage, and fused diff retrieval. On representative multi-agent workloads, it reduces end-to-end latency by up to 2.3× and KV Cache storage by 94% compared to vLLM with prefix caching.

More broadly, our work argues that communication structure should become a first-class concept in LLM serving. The All-Gather pattern is a natural starting point because it already appears across many agent workloads. As agent systems continue to diversify, other recurring communication patterns will likely expose similar opportunities. A communication-pattern-aware serving stack will be important for making future multi-agent platforms efficient at scale.

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 117–134.

[2] Zhuohang Bian, Feiyang Wu, Teng Ma, and Youwei Zhuo. 2025. Tokencake: A KV-Cache-centric Serving Framework for LLM-based Multi-Agent Applications. *arXiv preprint arXiv:2510.18586* (2025).

[3] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.

[4] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. arXiv:2403.19708 [cs.CL] https://arxiv.org/abs/2403.19708

[5] Shiwei Gao, Youmin Chen, and Jiwu Shu. 2025. Fast state restoration in llm serving with hcache. In *Proceedings of the Twentieth European Conference on Computer Systems*. 128–143.

[6] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2024. Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs. arXiv:2310.01801 [cs.CL] https://arxiv.org/abs/2310.01801

[7] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. De Sa (Eds.), Vol. 6.

325–338. https://proceedings.mlsys.org/paper_files/paper/2024/file/a66caa1703fe34705a4368c3014c1966-Paper-Conference.pdf

[8] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv:2308.00352 [cs.AI] https://arxiv.org/abs/2308.00352

[9] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 1270–1303. doi:10.52202/079017-0040

[10] Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Tiancheng Hu, Qin Zhang, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. 2024. EPIC: Efficient position-independent caching for serving large language models. *arXiv preprint arXiv:2410.15332* (2024).

[11] Hyesung Jeon, Hyeongju Ha, and Jae-Joon Kim. 2026. LRAgent: Efficient KV Cache Sharing for Multi-LoRA LLM Agents. arXiv:2602.01053 [cs.LG] https://arxiv.org/abs/2602.01053

[12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.

[13] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. {InfiniGen}: Efficient generative inference of large language models with dynamic {KV} cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 155–172.

[14] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 51991–52008. https://proceedings.neurips.cc/paper_files/paper/2023/file/a3621ee907def47c1b952ade25c67698-Paper-Conference.pdf

[15] Lehui Li, Ruining Wang, Haochen Song, Yaoxin Mao, Tong Zhang, Yuyao Wang, Jiayi Fan, Yitong Zhang, Jieping Ye, Chengqi Zhang, et al. 2026. What Papers Don't Tell You: Recovering Tacit Knowledge for Automated Paper Reproduction. *arXiv preprint arXiv:2603.01801* (2026).

[16] Zhonghang Li, Long Xia, Lei Shi, Yong Xu, Dawei Yin, and Chao Huang. 2024. Opencity: Open spatio-temporal foundation models for traffic prediction. *arXiv preprint arXiv:2408.10269* (2024).

[17] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 929–945.

[18] Yuhan Liu, Yuyang Huang, Jiayi Yao, Shaoting Feng, Zhuohan Gu, Kuntai Du, Hanchen Li, Yihua Cheng, Junchen Jiang, Shan Lu, Madan Musuvathi, and Esha Choukse. 2025. DroidSpeak: KV Cache Sharing for Cross-LLM Communication and Multi-LLM Serving. arXiv:2411.02820 [cs.MA] https://arxiv.org/abs/2411.02820

[19] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.

[20] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2023. Scissorhands: exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 2279, 23 pages.

[21] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen (Henry) Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: a tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) *(ICML'24)*. JMLR.org, Article 1311, 13 pages.

[22] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E Gonzalez, et al. 2025. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965* (2025).

[23] moltbook. 2026. moltbook: The front page of the agent internet. https://www.moltbook.com/. Website. Accessed: 2026-03-31.

[24] OpenClaw contributors. 2026. OpenClaw: Your own personal AI assistant. Any OS. Any Platform. The lobster way. https://github.com/openclaw/openclaw. GitHub repository. Accessed: 2026-03-31.

[25] Zaifeng Pan, Yipeng Shen, Zhengding Hu, Zhuang Wang, Aninda Manocha, Zheng Wang, Zhongkai Yu, Yue Guan, and Yufei Ding. 2026. ScaleSim: Serving Large-Scale Multi-Agent Simulation with Invocation Distance-Based Memory Management. *arXiv preprint arXiv:2601.21473* (2026).

[26] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*. 1–22.

[27] Jinghua Piao, Yuwei Yan, Jun Zhang, Nian Li, Junbo Yan, Xiaochong Lan, Zhihong Lu, Zhiheng Zheng, Jing Yi Wang, Di Zhou, et al. 2025. Agentsociety: Large-scale simulation of llm-driven generative agents advances understanding of human behaviors and society. (2025).

[28] Xavier Puig, Kevin Ra, Marko Boben, Jiaman Li, Tingwu Wang, Sanja Fidler, and Antonio Torralba. 2018. Virtualhome: Simulating household activities via programs. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 8494–8502.

[29] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 15174–15186. doi:10.18653/v1/2024.acl-long.810

[30] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, et al. 2024. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *ACM Transactions on Storage* (2024).

[31] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2024. Teola: Towards end-to-end optimization of llm-based applications. *arXiv preprint arXiv:2407.00326* (2024).

[32] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI] https://arxiv.org/abs/2308.08155

[33] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. arXiv:2309.17453 [cs.CL] https://arxiv.org/abs/2309.17453

[34] Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. 2025. Kvlink: Accelerating large language models via efficient kv cache reuse. *arXiv preprint arXiv:2502.16002* (2025).

[35] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. Cacheblend:

Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the twentieth European conference on computer systems*. 94–109.

[36] Hancheng Ye, Zhengqi Gao, Mingyuan Ma, Qinsi Wang, Yuzhe Fu, Ming-Yu Chung, Yueqian Lin, Zhijian Liu, Jianyi Zhang, Danyang Zhuo, et al. 2025. Kvcomm: Online cross-context kv-cache communication for efficient llm-based multi-agent systems. *arXiv preprint arXiv:2510.12872* (2025).

[37] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. 2025. Flashinfer: Efficient and customizable attention engine for llm inference serving. *Proceedings of Machine Learning and Systems* 7 (2025).

[38] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX symposium on operating systems design and implementation (OSDI 22)*. 521–538.

[39] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang "Atlas" Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 34661–34710. https://proceedings.neurips.cc/paper_files/paper/2023/file/6ceefa7b15572587b78ecfcebb2827f8-Paper-Conference.pdf

[40] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.